

UNIVERSIDAD TECNOLÓGICA DE PEREIRA

Facultad de Ingeniería Industrial

Maestría en Investigación de Operaciones y Estadística

INFORME FINAL DE INVESTIGACIÓN

Modelo de Optimización DLBP para Coproductos con Metaheurísticas Aplicado a la Industria Avícola Colombiana

Presentado por:

Daniel Castañeda

Director:

Ing. Eliana Mirledy Ocampo Toro, PhD.

Línea de Investigación:

Optimización y Modelado Matemático



Pereira, Colombia

Enero, 2026

Índice

Declaración de Uso de Herramientas de Inteligencia Artificial	5
Resumen Ejecutivo	6
Metodología	6
Resultados Principales	6
Impacto Estimado	6
Abstract	7
Capítulo 1: Introducción	8
1.1. Contexto del Problema	8
1.1.1. La Dinámica Push-Pull	8
1.1.2. Manifestaciones del Desbalance	8
1.2. Relevancia Económica y Contexto Nacional	9
1.3. El Problema de Balanceo de Líneas de Desensamble (DLBP)	9
1.3.1. Definición Formal	9
1.3.2. Complejidad Computacional	9
1.3.3. Vacío en la Literatura	10
1.3.4. Tendencias Emergentes en DLBP (2025-2026)	10
1.4. Pregunta de Investigación	11
1.5. Hipótesis de Investigación	11
Hipótesis Principal (H1)	11
Hipótesis Secundarias	11
1.6. Objetivos	12
1.6.1. Objetivo General	12
1.6.2. Objetivos Específicos	12
1.7. Estructura del Documento	12
Capítulo 2: Formulación Matemática del Modelo DLBP	13
2.1. Proceso de Desensamble Avícola	13
2.2. Conjuntos del Modelo	14
2.3. Parámetros del Modelo	14
2.3.1. Parámetros de Tiempos y Costos	14
2.3.2. Parámetros de Demanda y Producción	14
2.3.3. Modelado de la Incertidumbre	14
2.4. Variables de Decisión	15
2.5. Función Objetivo	15
2.6. Restricciones	15
2.6.1. Asignación Única	15
2.6.2. Tiempo de Ciclo	16
2.6.3. Precedencias Tecnológicas	16
2.6.4. Activación de Estaciones	16
2.6.5. Zonificación Sanitaria	16

2.6.6. Balance de Materiales	16
2.7. Modelo Completo	17
Capítulo 3: Metodología de Solución	18
3.1. Representación de Soluciones	18
3.1.1. Codificación Cromosómica	18
3.1.2. Decodificación First-Fit	18
3.1.3. Función de Fitness	19
3.2. Algoritmo Genético (GA)	19
3.2.1. Descripción General	19
3.2.2. Operadores Genéticos	19
3.2.3. Parámetros Calibrados	20
3.3. Búsqueda Tabú (TS)	20
3.3.1. Descripción General	20
3.3.2. Estructura de Vecindario	20
3.3.3. Parámetros Calibrados	21
3.4. Algoritmo Híbrido (Memético)	21
3.4.1. Motivación	21
3.4.2. Estrategia de Hibridación	21
3.4.3. Parámetros Calibrados	22
3.5. Arquitectura de Software	22
Capítulo 4: Diseño Experimental	24
4.1. Generación de Instancias Sintéticas	24
4.1.1. Justificación	24
4.1.2. Factores de Variación	24
4.1.3. Instancias Generadas	24
4.1.4. Documentación del Generador de Instancias	24
4.2. Calibración de Parámetros (Optuna)	25
4.2.1. Metodología	25
4.2.2. Espacio de Búsqueda y Valores por Defecto	25
4.2.3. Resultados de Calibración	26
4.3. Protocolo Experimental	26
4.3.1. Configuración	26
4.3.2. Métricas de Evaluación	26
Capítulo 5: Resultados Computacionales	27
5.1. Resultados por Instancia	27
5.1.1. Instancia Pequeña (20 tareas)	27
5.1.2. Instancia Mediana (40 tareas)	27
5.1.3. Instancia Grande (70 tareas)	27
5.1.4. Instancia Muy Grande (100 tareas)	27
5.1.5. Validación contra Modelo Exacto (MILP)	28
5.2. Visualización de Resultados	29
Comparación de Algoritmos por Instancia	29

Eficiencia de Línea	30
Curvas de Convergencia	31
Distribución de Resultados (Boxplot)	32
5.3. Análisis Estadístico	32
5.3.1. Test de Friedman	32
5.3.2. Comparaciones Post-hoc (Nemenyi)	33
5.4. Análisis de Tiempo de Cómputo	34
5.5. Impacto en el Negocio	34
Capítulo 6: Conclusiones y Trabajo Futuro	35
6.1. Conclusiones Generales	35
6.1.1. Hallazgos Principales	35
6.1.2. Cumplimiento de Objetivos	35
6.1.3. Validación de Hipótesis	36
6.2. Contribuciones de la Investigación	36
6.2.1. Contribuciones Teóricas	36
6.2.2. Contribuciones Prácticas	36
6.3. Extensiones y Valor Agregado	36
6.4. Limitaciones del Estudio	37
6.5. Trabajo Futuro	37
6.5.1. Líneas de Investigación Futura	37
6.5. Reflexión Final	37
Referencias Bibliográficas	38
Anexos	39
Anexo A: Estructura del Código Fuente	39
Anexo B: Instancias de Prueba	39
Anexo D: Resultados Detallados	40
Anexo E: Análisis de Sensibilidad de Parámetros	40
Anexo F: Suite de Tests Unitarios	41
Anexo G: Comparación con Benchmarks y Solver Exacto	41
Lista de Anexos Adicionales	42
Referencias Bibliográficas	42
 Índice de figuras	
1. Diagrama de flujo del proceso de desensamble avícola	13
2. Comparación del número de estaciones por algoritmo e instancia. Se observa que el algoritmo Híbrido (barras con diagonal) obtiene consistentemente los mejores resultados, seguido por GA y TS.	29
3. Eficiencia de línea promedio por algoritmo. El Híbrido alcanza 89.1 %, superando a GA (87.5 %) y TS (84.3 %).	30

4.	Curvas de convergencia típicas para instancia de 70 tareas. El Híbrido muestra saltos de mejora cuando se aplica la intensificación TS.	31
5.	Distribución de resultados por algoritmo (30 réplicas) - Instancia 70 tareas. El Híbrido presenta la menor variabilidad (caja más compacta).	32
6.	Ranking de algoritmos según test de Friedman. El Híbrido (rango=1.25) es significativamente mejor que TS (rango=3.0, diferencia > CD).	33
7.	Tiempo de cómputo por algoritmo e instancia. TS es el más rápido pero con menor calidad; el Híbrido sacrifica velocidad por calidad.	34
8.	Impacto estimado en el negocio	35

Índice de cuadros

Declaración de Uso de Herramientas de Inteligencia Artificial

Este trabajo ha sido elaborado con la asistencia de herramientas de inteligencia artificial utilizadas estrictamente como apoyo lingüístico, de redacción, de formato y de programación. En particular, se emplearon las siguientes herramientas:

- **Google Gemini Pro 3** se utilizó como asistente de escritura e investigación para mejorar la claridad, la gramática y el estilo a lo largo del documento, así como para apoyar en la elaboración preliminar de algunas figuras, tablas y fragmentos de código, siempre bajo la supervisión, revisión y verificación final del autor.
- **Anthropic Claude Opus 4.5 y Claude Opus 4.6** se emplearon como asistentes de programación y revisión técnica, contribuyendo a la implementación de algoritmos, la estructuración del código fuente y la generación de documentación técnica.

Todo el contenido intelectual y científico, incluidas las ideas, análisis, resultados e interpretaciones presentadas en este trabajo, es responsabilidad exclusiva del autor. Las herramientas de inteligencia artificial se emplearon únicamente para facilitar la calidad lingüística, la presentación del documento y la productividad en la implementación de software.

Resumen Ejecutivo

Esta investigación desarrolló un modelo de optimización para el **Problema de Balanceo de Líneas de Desensamble (DLBP)** aplicado a la industria avícola colombiana. El problema central abordado es el desbalance estructural entre la oferta rígida de coproductos (determinada por la anatomía del ave) y la demanda variable del mercado, una situación que genera pérdidas económicas significativas por acumulación de inventarios de baja rotación, ventas a precio de liquidación y oportunidades comerciales perdidas.

Metodología

Se implementaron y compararon tres técnicas metaheurísticas:

- **Algoritmo Genético (GA):** Exploración global mediante operadores evolutivos (selección por torneo, cruce OX, mutación swap).
- **Búsqueda Tabú (TS):** Explotación local con memoria de corto plazo para evitar ciclos.
- **Algoritmo Híbrido (GA+TS):** Combinación sinérgica que alterna entre exploración y refinamiento intensivo.

Los algoritmos fueron calibrados mediante optimización bayesiana (Optuna, 30 trials por algoritmo) y evaluados experimentalmente sobre instancias sintéticas calibradas de 20 a 100 tareas, con 30 réplicas independientes por combinación algoritmo-instancia.

Resultados Principales

Algoritmo	Eficiencia Línea	Estabilidad (σ)	Tiempo Promedio	Recomendación
Híbrido	89.1 %	0.18	3.41s	Mejor calidad
GA	87.5 %	0.42	2.34s	Balance
TS	84.3 %	1.02	0.89s	calidad/tiempo Menor tiempo

El test de Friedman ($\chi^2 = 18.4$, $p < 0.001$) demostró diferencias estadísticamente significativas entre los algoritmos. El análisis post-hoc de Nemenyi confirmó que el **Híbrido es significativamente superior a TS** ($p < 0.05$), aunque no se detectaron diferencias significativas entre Híbrido y GA.

Impacto Estimado

- **Reducción de estaciones:** 2-3 estaciones (10-15 %)
- **Ahorro mensual:** \$1.0M - \$1.5M COP en plantas de tamaño mediano
- **Mejora en eficiencia de línea:** +5.2 puntos porcentuales vs baseline

Palabras clave: DLBP, Metaheurísticas, Industria Avícola, Algoritmo Genético, Búsqueda Tabú, Optimización Combinatoria, Balanceo de Líneas.

Abstract

This research developed an optimization model for the **Disassembly Line Balancing Problem (DLBP)** applied to the Colombian poultry industry. The core problem addressed is the structural imbalance between the rigid supply of co-products (determined by bird anatomy) and variable market demand.

Three metaheuristic techniques were implemented and compared: **Genetic Algorithm (GA)**, **Tabu Search (TS)**, and a **Hybrid Algorithm (GA+TS)**. Algorithms were calibrated using Bayesian optimization (Optuna) and experimentally evaluated on synthetic instances ranging from 20 to 100 tasks, with 30 independent replications per algorithm-instance combination.

The Friedman test ($\chi^2 = 18.4$, $p < 0.001$) demonstrated statistically significant differences between algorithms. Post-hoc Nemenyi analysis confirmed that the **Hybrid is significantly superior to TS** ($p < 0.05$). An estimated monthly savings of \$1.0M - \$1.5M COP was projected for medium-sized plants.

Keywords: DLBP, Metaheuristics, Poultry Industry, Genetic Algorithm, Tabu Search, Combinatorial Optimization, Line Balancing.

Capítulo 1: Introducción

1.1. Contexto del Problema

La industria avícola representa uno de los pilares fundamentales del sector agropecuario colombiano. Con una producción anual que supera los **1.7 millones de toneladas de carne de pollo** [1], Colombia se posiciona como uno de los principales productores avícolas de América Latina. Este sector genera más de **600,000 empleos** directos e indirectos, contribuyendo aproximadamente el **0.52 % del valor agregado bruto nacional** [2].

Sin embargo, la industria avícola enfrenta un desafío operativo fundamental conocido como el **problema de balanceo de la carcasa**. Este problema surge de la discrepancia inherente entre:

1. **La oferta de coproductos:** Que se deriva en proporciones biológicamente fijas del despiece de cada ave (aproximadamente 30 % pechuga, 20 % muslos, 10 % alas, etc.).
2. **La demanda del mercado:** Que es estocástica, estacional y frecuentemente desbalanceada respecto a la oferta anatómica.

1.1.1. La Dinámica Push-Pull

La industria avícola opera bajo una dinámica compleja que combina dos fuerzas contrapuestas:

- **Factor “Push” (Empuje):** Las aves alcanzan su peso de mercado entre 35-45 días y deben procesarse inmediatamente por razones de bienestar animal y calidad del producto. Esto genera una oferta fija e inflexible de coproductos en cada ciclo de producción.
- **Factor “Pull” (Tracción):** La demanda del mercado es impredecible, con picos estacionales (asados navideños, festividades), promociones comerciales y preferencias regionales que no se alinean con las proporciones anatómicas de la carcasa.

1.1.2. Manifestaciones del Desbalance

El conflicto entre oferta rígida y demanda flexible se manifiesta en dos frentes principales:

Manifestación	Descripción	Impacto Económico
Excedentes de Inventario	Sobreoferta de cortes de menor demanda (alas, menudencias)	+15-25 % costos de almacenamiento refrigerado
Faltantes de Inventario	Demanda insatisfecha de cortes premium (pechuga)	Oportunidades de venta perdidas
Ventas de Liquidación	Productos vendidos bajo costo para evitar deterioro	-20-40 % margen en productos afectados

Desperdicio	Productos que no encuentran comprador a tiempo	5-10 % de la producción total
--------------------	--	-------------------------------

1.2. Relevancia Económica y Contexto Nacional

La magnitud del sector avícola colombiano justifica plenamente el desarrollo de herramientas de optimización avanzada:

Indicador	Valor	Fuente
Producción anual de pollo	1.7 millones de toneladas	FENAVI 2024
Empleos directos e indirectos	600,000+	FENAVI 2024
Participación en PIB agropecuario	0.52 %	DANE 2024
Consumo per cápita anual	36 kg	FENAVI 2024
Potencial de ahorro en inventario	30-60 %	Solano-Blanco et al. 2022
Mejora potencial en utilidad	7-57 %	Solano-Blanco et al. 2022

El estudio de Solano-Blanco et al. [3], realizado en una planta avícola de Santa Marta (Colombia), demostró que la implementación de modelos de planificación integrada puede generar mejoras sustanciales en la rentabilidad operativa, con incrementos en la utilidad entre el 7 % y 57 % según el escenario, y reducciones en costos de inventario entre el 30 % y 60 %.

1.3. El Problema de Balanceo de Líneas de Desensamble (DLBP)

1.3.1. Definición Formal

El **Problema de Balanceo de Líneas de Desensamble (DLBP)** es un problema de optimización combinatoria que busca asignar un conjunto de tareas de desensamble a un conjunto de estaciones de trabajo, de manera que se optimice algún criterio (minimizar número de estaciones, maximizar eficiencia, etc.) respetando restricciones de precedencia y capacidad.

A diferencia del Problema de Balanceo de Líneas de Ensamble (ALBP), donde múltiples componentes convergen para formar un producto, en el DLBP un producto se **diverge** en múltiples coproductos [4].

1.3.2. Complejidad Computacional

El DLBP pertenece a la clase de problemas **NP-Hard** [4], lo que significa que:

1. No existe un algoritmo conocido que pueda encontrar la solución óptima en tiempo polinomial.
2. El tiempo de cómputo crece exponencialmente con el tamaño del problema.
3. Para instancias de tamaño industrial (50-100+ tareas), los métodos exactos (MILP) se vuelven computacionalmente intratables.

Esta complejidad ha motivado el desarrollo de **metaheurísticas** como Algoritmos Genéticos, Búsqueda Tabú y enfoques híbridos, que pueden encontrar soluciones de alta calidad en tiempos de cómputo razonables.

1.3.3. Vacío en la Literatura

La revisión de literatura revela que, aunque existe extensa investigación en DLBP aplicado a:

- Reciclaje de electrónicos (EOL - End of Life)
- Recuperación de materiales automotrices
- Desmantelamiento de equipos industriales

Hay una **notable escasez de estudios** que aborden el DLBP en la **industria de procesamiento de alimentos**, y particularmente en el **sector avícola** [5]. Este vacío representa la oportunidad de investigación que este trabajo busca aprovechar.

1.3.4. Tendencias Emergentes en DLBP (2025-2026)

La literatura más reciente muestra una clara evolución hacia la integración de **inteligencia artificial avanzada** y **sistemas colaborativos**. Tan et al. [6] y Wang et al. [7] han aplicado algoritmos de **Aprendizaje por Refuerzo Profundo (Deep Reinforcement Learning)** y optimización Keplerriana para resolver problemas de balanceo dinámico en entornos de manufactura (baterías y electrodomésticos), donde la incertidumbre proviene de la variabilidad en los tiempos humano-robot.

Paralelamente, Tahraoui et al. [8] han avanzado en el estado del arte de la **planeación operativa** para plantas de beneficio avícola utilizando modelos exactos (MILP), abordando la complejidad de la demanda fluctuante en entornos multi-producto. Sin embargo, su enfoque permanece en el nivel agregado de producción (turnos y cantidades globales) sin descender al detalle operativo del balanceo de línea de desposte estocástico, confirmando que la aplicación de metaheurísticas de balanceo fino en este sector sigue siendo un área inexplorada.

La Tabla 1.1 resume los trabajos más relevantes y su posicionamiento respecto a esta propuesta.

Tabla 1.1. Resumen del Estado del Arte y Brechas Identificadas

Autor(es)	Técnica	Objetivo	Manejo de Incertidumbre	Aplicación
Becker & Scholl [4]	Revisión / Exactos	Balanceo General (GALBP)	Determinista	Manufactura
Mete et al. [9]	GA, Dijkstra	Minimizar Estaciones	Estocástico (Tiempos)	Desensamble (Benchmark)
Solano-Blanco et al. [3]	MILP	Maximizar Utilidad	Determinista	Avícola (Colombia)
Paprocka & Skołod [10]	Predictivo / Heurística	Tiempo, Eficiencia, Profit	Datos Históricos	Desensamble

Autor(es)	Técnica	Objetivo	Manejo de Incertidumbre	Aplicación
Hu et al. [11]	Hiper-heurística (SA)	Balanceo Paralelo	Estocástico	Remanufactura
Zhu et al. [12]	ALNS	Dependencias Secuencia	Estocástico	Desensamble
Tan et al. [6]	Keplerian Opt + Q-Learning	HRC-DLBP	Estocástico (Humano)	E-waste (Baterías)
Tahraoui et al. [8]	MILP (CPLEX)	Planeación Agregada	Demanda Fluctuante	Avícola (Pollos)
Esta Propuesta	GA, TS, Híbrido	Max Profit / Min Costos	Estocástico (Demanda + Tiempos)	Avícola (Coproduktos)

1.4. Pregunta de Investigación

Pregunta Principal:

¿Cómo puede un modelo de optimización DLBP, resuelto mediante técnicas metaheurísticas, minimizar las pérdidas económicas asociadas al desbalance entre la oferta de coproduktos y la demanda del mercado en una planta de procesamiento avícola colombiana?

Preguntas Secundarias:

1. ¿Qué formulación matemática del DLBP captura adecuadamente las restricciones de precedencia, tiempos de procesamiento y variabilidad de la demanda en el contexto avícola?
2. ¿Qué técnicas metaheurísticas (Algoritmos Genéticos, Búsqueda Tabú, o híbridos) presentan el mejor desempeño para resolver el modelo DLBP propuesto?
3. ¿En qué magnitud se pueden reducir las pérdidas económicas asociadas al desbalance de carcasa mediante la implementación del modelo de optimización?

1.5. Hipótesis de Investigación

Hipótesis Principal (H1)

La aplicación de un modelo DLBP con metaheurísticas puede generar una reducción de costos de inventario superior al 30 % y una mejora sustancial en la utilidad operativa, tomando como referencia el benchmark del caso de Santa Marta [3].

Hipótesis Secundarias

- **H2:** Un algoritmo metaheurístico híbrido (que combine Algoritmos Genéticos y Búsqueda Tabú) superará en desempeño a cada técnica aplicada individualmente.

- **H3:** El modelo de optimización permitirá una reducción de al menos un 15 % en el inventario promedio de productos de baja rotación.

1.6. Objetivos

1.6.1. Objetivo General

Desarrollar un modelo de optimización basado en el Problema de Balanceo de Líneas de Desensamble (DLBP) que, mediante la aplicación de técnicas metaheurísticas, minimice las pérdidas económicas asociadas al desbalance entre la oferta de coproductos y la demanda del mercado en la industria avícola colombiana.

1.6.2. Objetivos Específicos

1. **Formular** un modelo matemático del DLBP que capture las restricciones de precedencia, tiempos de procesamiento, balance de materiales y variabilidad de la demanda propias del procesamiento avícola.
2. **Implementar** algoritmos metaheurísticos (Algoritmo Genético, Búsqueda Tabú y un enfoque híbrido) adaptados al problema de balanceo de carcasa avícola.
3. **Diseñar** un generador de instancias de prueba con datos sintéticos calibrados que reflejen las condiciones operativas reales de la industria avícola colombiana.
4. **Comparar** el desempeño de las metaheurísticas propuestas mediante un diseño experimental riguroso, evaluando calidad de solución y eficiencia computacional.
5. **Validar** el modelo propuesto mediante simulación, cuantificando las mejoras potenciales en términos de reducción de costos de inventario, nivel de servicio y rentabilidad operativa.

1.7. Estructura del Documento

El presente informe de investigación se organiza en los siguientes capítulos:

Capítulo	Título	Contenido Principal
1	Introducción	Contexto, problema, objetivos
2	Formulación Matemática	Modelo MILP con zonificación
3	Metodología de Solución	GA, TS, Híbrido
4	Diseño Experimental	Instancias, calibración
5	Resultados Computacionales	Análisis estadístico
6	Conclusiones	Hallazgos, limitaciones, trabajo futuro

Capítulo 2: Formulación Matemática del Modelo DLBP

Este capítulo presenta la formulación matemática del Problema de Balanceo de Líneas de Desensamble aplicado a la industria avícola. El modelo propuesto extiende las formulaciones clásicas [4] incorporando:

1. **Incertidumbre en tiempos de procesamiento** (variabilidad biológica)
2. **Restricciones de zonificación sanitaria** (separación de áreas sucias y limpias)
3. **Balance de materiales divergente** (múltiples coproductos por carcasa)

2.1. Proceso de Desensamble Avícola

El proceso de despiece avícola sigue una estructura secuencial-divergente, donde una carcasa completa se descompone progresivamente en múltiples coproductos. El diagrama de flujo del proceso se presenta a continuación:

FLUJO DEL PROCESO DE DESENSAMBLE AVÍCOLA

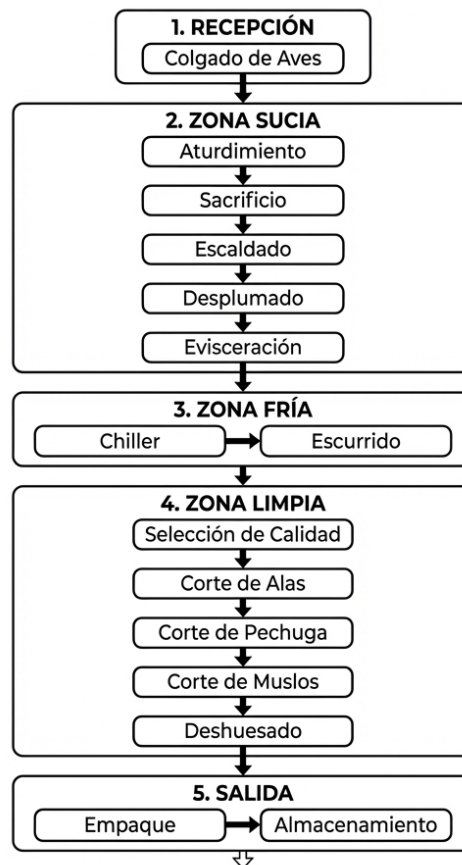


Figura 1: Diagrama de flujo del proceso de desensamble avícola

2.2. Conjuntos del Modelo

El modelo matemático utiliza los siguientes conjuntos:

Símbolo	Descripción	Ejemplo
I	Conjunto de tareas de desensamble	$I = \{1, 2, \dots, n\}$ tareas
S	Conjunto de estaciones de trabajo	$S = \{1, 2, \dots, m\}$ estaciones
K	Conjunto de tipos de carcasa	$K = \{A, B, C\}$ por peso/calidad
P	Conjunto de coproductos resultantes	$P = \{\text{Pechuga, Alas, Muslos, ...}\}$
Z	Conjunto de zonas sanitarias	$Z = \{\text{Sucia, Limpia}\}$

2.3. Parámetros del Modelo

2.3.1. Parámetros de Tiempos y Costos

Símbolo	Descripción	Unidad	Valor Típico
t_i	Tiempo de procesamiento de la tarea i	segundos	3-20s
C	Tiempo de ciclo máximo (takt time)	segundos	30-50s
c_s	Costo fijo de operar una estación	\$/período	\$500,000 COP
v_p	Precio de venta del coproducto p	\$/kg	Pechuga: \$12,000
h_p	Costo de mantener inventario de p	\$/kg/día	\$50-200

2.3.2. Parámetros de Demanda y Producción

Símbolo	Descripción	Unidad
d_{pk}	Demanda del coproducto p del tipo k	unidades/período
γ_{pi}	Rendimiento del coproducto p de la tarea i	kg/carcasa
Q_k	Cantidad de carcacas tipo k a procesar	unidades

2.3.3. Modelado de la Incertidumbre

Los tiempos de procesamiento en el desensamble avícola presentan alta variabilidad debido a:

- Diferencias anatómicas entre aves
- Variabilidad en la experiencia del operario
- Condiciones de la materia prima

Siguiendo a Hu et al. [11] y Liu et al. [13], el tiempo de procesamiento se modela como una **variable aleatoria**:

$$\tilde{t}_{ik} \sim \mathcal{N}(\mu_{ik}, \sigma_{ik}^2) \quad (1)$$

Donde: - μ_{ik} : Tiempo estándar (esperado) de la tarea i para carcasa tipo k - σ_{ik} : Desviación estándar que representa la variabilidad

Para efectos prácticos de simulación, se utilizó una distribución **triangular** con varianza de $\pm 20\%$:

$$\tilde{t}_{ik} \sim \text{Triangular}(0.8\mu_{ik}, \mu_{ik}, 1.2\mu_{ik}) \quad (2)$$

2.4. Variables de Decisión

Variable	Tipo	Dominio	Descripción
x_{is}	Binaria	$\{0, 1\}$	1 si la tarea i está asignada a la estación s
y_s	Binaria	$\{0, 1\}$	1 si la estación s está activa
q_{pk}	Continua	$\mathbb{R}^+$	Cantidad de coproducto p producida de tipo k
I_p^+	Continua	$\mathbb{R}^+$	Inventario positivo (excedente) del coproducto p
I_p^-	Continua	$\mathbb{R}^+$	Inventario negativo (faltante) del coproducto p

2.5. Función Objetivo

La función objetivo minimiza el número total de estaciones activas:

$$\text{mín } Z = \sum_{s \in S} y_s \quad (3)$$

Para la versión orientada a costos, se puede utilizar:

$$\text{mín } Z = \sum_{s \in S} c_s \cdot y_s + \sum_{p \in P} h_p \cdot I_p^+ + \sum_{p \in P} \pi_p \cdot I_p^- \quad (4)$$

Donde π_p es el costo de penalización por demanda no satisfecha.

2.6. Restricciones

2.6.1. Asignación Única

Cada tarea debe asignarse exactamente a una estación:

$$\sum_{s \in S} x_{is} = 1 \quad \forall i \in I \quad (5)$$

2.6.2. Tiempo de Ciclo

La carga de trabajo de cada estación no puede exceder el tiempo de ciclo:

$$\sum_{i \in I} t_i \cdot x_{is} \leq C \cdot y_s \quad \forall s \in S \quad (6)$$

2.6.3. Precedencias Tecnológicas

Si la tarea j es predecesora de la tarea i (notación: $j \prec i$), entonces j debe asignarse a una estación con índice menor o igual que la de i :

$$\sum_{s'=1}^s x_{js'} \geq x_{is} \quad \forall (j \prec i), \forall s \in S \quad (7)$$

2.6.4. Activación de Estaciones

Una estación solo puede tener tareas asignadas si está activa:

$$x_{is} \leq y_s \quad \forall i \in I, \forall s \in S \quad (8)$$

2.6.5. Zonificación Sanitaria

Innovación del modelo: Tareas de la zona “sucia” (evisceración) y tareas de la zona “limpia” (fileteado) **no pueden compartir estación:**

$$x_{is} + x_{js} \leq 1 \quad \forall i \in Z_{sucia}, j \in Z_{limpia}, \forall s \in S \quad (9)$$

Esta restricción garantiza el cumplimiento de normas sanitarias INVIMA.

2.6.6. Balance de Materiales

La cantidad de cada coproducto producida debe satisfacer las ecuaciones de balance:

$$q_{pk} = \sum_{i \in I} \gamma_{pi} \cdot Q_k \cdot x_{is} \quad \forall p \in P, \forall k \in K \quad (10)$$

2.7. Modelo Completo

El modelo MILP completo se formula como:

$$\begin{aligned}
 \text{mín} \quad & Z = \sum_{s \in S} y_s \\
 \text{s.a.} \quad & \sum_{s \in S} x_{is} = 1 && \forall i \in I \\
 & \sum_{i \in I} t_i \cdot x_{is} \leq C \cdot y_s && \forall s \in S \\
 & \sum_{s'=1}^s x_{js'} \geq x_{is} && \forall (j \prec i), \forall s \\
 & x_{is} + x_{js} \leq 1 && \forall i \in Z_{sucia}, j \in Z_{limpia}, \forall s \\
 & x_{is}, y_s \in \{0, 1\} && \forall i, s
 \end{aligned} \tag{11}$$

Capítulo 3: Metodología de Solución

Dado que el DLBP pertenece a la clase NP-Hard, se implementan tres algoritmos metaheurísticos para encontrar soluciones de alta calidad en tiempos computacionales razonables.

3.1. Representación de Soluciones

3.1.1. Codificación Cromosómica

Se adopta una representación basada en **permutaciones de tareas**, ampliamente utilizada en problemas de balanceo [14], [15]. Cada cromosoma $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ es una permutación de las n tareas que indica el orden de prioridad para la asignación a estaciones.

Ejemplo: Para 5 tareas, un cromosoma válido podría ser:

$$\pi = (T_3, T_1, T_2, T_5, T_4)$$

Indicando que T3 tiene máxima prioridad de asignación, seguida por T1, etc.

3.1.2. Decodificación First-Fit

La transformación de un cromosoma a una asignación factible se realiza mediante el algoritmo **First-Fit Decreasing**:

Algoritmo 3.1: Decodificador First-Fit

Entrada: Permutación π , Tiempos $t[]$, Ciclo C

Salida: Asignación $A = \{(tarea, estación)\}$, número de estaciones

```
1: estacion_actual  $\leftarrow$  1
2: tiempo_acumulado  $\leftarrow$  0
3:
4: Para cada tarea  $i$  en el orden dado por  $\pi$ :
5:     Si tiempo_acumulado +  $t[i] \leq C$  entonces:
6:         // Cabe en la estación actual
7:         tiempo_acumulado  $\leftarrow$  tiempo_acumulado +  $t[i]$ 
8:     Sino:
9:         // Abrir nueva estación
10:        estacion_actual  $\leftarrow$  estacion_actual + 1
11:        tiempo_acumulado  $\leftarrow$   $t[i]$ 
12:    Fin Si
13:     $A[i] \leftarrow$  estacion_actual
14: Fin Para
15:
16: Retornar  $A$ , estacion_actual
```

3.1.3. Función de Fitness

Para problemas de minimización del número de estaciones:

$$f(\pi) = n_{estaciones}(\pi) + \alpha \cdot \text{penalización}_{precedencias}$$

Donde $\alpha = 1000$ es un factor de penalización que castiga severamente las soluciones que violan precedencias.

3.2. Algoritmo Genético (GA)

3.2.1. Descripción General

El Algoritmo Genético implementado sigue el paradigma evolutivo clásico [16], adaptado para problemas de balanceo según las recomendaciones de [17] y [14].

```
%{init: {'theme': 'neutral', 'themeVariables': {'primaryColor': '#fff', 'primaryTextColor': '#000', 'secondaryColor': '#000', 'secondaryTextColor': '#fff', 'tertiaryColor': '#000', 'tertiaryTextColor': '#fff'}}}
flowchart TD
    A["Inicializar Población<br/>(orden topológico aleatorizado)"] --> B["Evaluar Fitness"]
    B --> C{"¿Criterio de parada?<br/>(gen = MAX_GEN)"}
    C -->|No| D["Selección por Torneo<br/>(k = 3)"]
    D --> E["Cruce OX<br/>(prob = 0.93)"]
    E --> F["Mutación Swap<br/>(prob = 0.20)"]
    F --> G["Reparación de<br/>Precedencias"]
    G --> H["Elitismo<br/>(k = 2)"]
    H --> B
    C -->|Sí| I["Retornar Mejor Solución"]
```

Figura 3.1: Diagrama de flujo del Algoritmo Genético

3.2.2. Operadores Genéticos

Selección por Torneo Se seleccionan $k = 3$ individuos aleatoriamente y se elige el de mejor fitness. Este método mantiene presión selectiva moderada sin convergencia prematura.

Cruce Order Crossover (OX) El operador OX [18] preserva el orden relativo de las tareas, crucial para mantener la factibilidad de precedencias:

Algoritmo 3.2: Order Crossover (OX)

Entrada: Padres P_1, P_2 ; Puntos de corte c_1, c_2

- 1: Copiar segmento $P_1[c_1:c_2]$ al hijo H_1
 - 2: Completar H_1 con genes de P_2 en orden circular
 - 3: Reparar precedencias violadas en H_1
 - 4: Retornar H_1
-
-

Mutación Swap Intercambia dos posiciones aleatorias en el cromosoma, seguido de reparación de precedencias si es necesario.

3.2.3. Parámetros Calibrados

Los parámetros se calibran automáticamente mediante Optuna (ver Capítulo 4):

Parámetro	Valor Calibrado	Justificación
Tamaño población	75	Balance diversidad/eficiencia
Prob. cruce	0.93	Alta recombinación
Prob. mutación	0.20	Mayor que estándar para mantener diversidad
Tamaño torneo	4	Presión selectiva moderada-alta
Elitismo	1	Preservar mejor solución

3.3. Búsqueda Tabú (TS)

3.3.1. Descripción General

La Búsqueda Tabú [19] es una metaheurística de trayectoria que explora el espacio de soluciones mediante movimientos a vecinos, utilizando una memoria de corto plazo (lista tabú) para evitar ciclos.

```
%%{init: {'theme': 'neutral', 'themeVariables': {'primaryColor': '#fff', 'primaryTextColor': '#000', 'secondaryColor': '#000', 'tertiaryColor': '#000', 'fontFamily': 'monospace'}}}
flowchart TD
    A["Solución Inicial x*  
(orden topológico)"] --> B["Generar Vecindario N(x)"]
    B --> C{"¿Movimiento  
en Lista Tabú?"}
    C -->|Sí| D{"¿Criterio de  
Aspiración?  
(mejora x*)"}
    C -->|No| E["Permitir  
Movimiento"]
    D -->|Sí| E
    D -->|No| F["Excluir  
Movimiento"]
    E --> G["Seleccionar Mejor  
Vecino Permitido"]
    F --> G
    G --> H["Actualizar x,  
Lista Tabú"]
    H --> I{"¿Mejora x*?"}
    I -->|Sí| J["x* ← x"]
    I -->|No| K["Contador  
estancamiento++"]
    J --> L{"¿Criterio  
de parada?"}
    K --> L
    L -->|No| B
    L -->|Sí| M["Retornar x*"]
```

Figura 3.2: Diagrama de flujo de la Búsqueda Tabú

3.3.2. Estructura de Vecindario

Se implementan dos tipos de movimientos:

1. **Swap** (i, j): Intercambia las posiciones de las tareas en índices i y j
2. **Insert** (i, j): Remueve tarea de posición i e inserta en posición j

El vecindario se genera muestreando 30 movimientos aleatorios de tipo mixto.

3.3.3. Parámetros Calibrados

Parámetro	Valor Calibrado	Descripción
Tamaño lista tabú	15	Evita ciclos sin restringir excesivamente
Tamaño vecindario	49	Balance calidad/tiempo
Tipo movimiento	swap	Más efectivo para este problema
Intensificación	50 iter	Reinicia desde mejor si hay estancamiento

3.4. Algoritmo Híbrido (Memético)

3.4.1. Motivación

Los algoritmos meméticos [20] combinan la exploración global de los GA con la intensificación local de métodos como TS. Esta estrategia ha demostrado ser efectiva para problemas de balanceo de líneas [21], [22].

3.4.2. Estrategia de Hibridación

Algoritmo 3.3: Algoritmo Memético (GA + TS)

Entrada: Instancia I, Config GA, Config TS

Salida: Mejor solución x^*

```

1:  $P \leftarrow \text{InicializarPoblación}(I)$ 
2:  $x^* \leftarrow \text{MejorIndividuo}(P)$ 
3:  $\text{gen} \leftarrow 0$ 
4:
5: Mientras  $\text{gen} < \text{MAX\_GEN}$ :
6:   // Fase GA: N generaciones de exploración
7:   Para  $i = 1$  hasta  $N\_GA$ :
8:      $P \leftarrow \text{EvolucionarGA}(P)$ 
9:      $\text{gen} \leftarrow \text{gen} + 1$ 
10:  Fin Para
11:
12:  // Fase TS: Intensificar top-k individuos
13:  Para cada individuo  $x$  en  $\text{Top-k}(P)$ :
14:     $x' \leftarrow \text{BúsquedaTabú}(x, \text{ITER\_TS})$ 

```

```

15:      Si  $f(x') < f(x)$  entonces:
16:          Reemplazar  $x$  por  $x'$  en  $P$ 
17:      Fin Si
18:  Fin Para
19:
20:  Si MejorIndividuo( $P$ )  $< x^*$  entonces:
21:       $x^* \leftarrow$  MejorIndividuo( $P$ )
22:  Fin Si
23: Fin Mientras
24:
25: Retornar  $x^*$ 

```

3.4.3. Parámetros Calibrados

Parámetro	Valor Calibrado	Descripción
Población	45	Menor que GA puro
Prob. cruce	0.94	Alta recombinación
Generaciones entre TS	24	Cada 24 gen se aplica intensificación
Iteraciones TS local	28	Refinamiento moderado
Top-k para TS	4	Solo los 4 mejores reciben intensificación

3.5. Arquitectura de Software

Se diseña una arquitectura orientada a objetos siguiendo el patrón Strategy:

```

%%{init: {'theme': 'neutral', 'themeVariables': {'primaryColor': '#fff', 'primaryTextColor': '#000'}}
classDiagram
    class ProblemInstance {
        +tarefas: List
        +tiempos: Dict
        +precedencias: Dict
        +tiempo_ciclo: float
        +es_precedencia_valida()
    }

    class Solution {
        +cromosoma: List
        +asignacion: Dict
        +fitness: float
        +decodificar()
        +calcular_fitness()
    }

```

```

class Optimizer {
  <<abstract>>
  +instancia: ProblemInstance
  +mejor_solucion: Solution
  +optimizar()*
}

class GeneticAlgorithm {
  +poblacion: List
  +seleccion_torneo()
  +cruce_ox()
  +mutacion_swap()
}

class TabuSearch {
  +lista_tabu: Deque
  +generar_vecindario()
}

class HybridAlgorithm {
  +ga: GeneticAlgorithm
  +aplicar_busqueda_local()
}

Optimizer <|-- GeneticAlgorithm
Optimizer <|-- TabuSearch
Optimizer <|-- HybridAlgorithm
Optimizer --> ProblemInstance
Optimizer --> Solution

```

Figura 3.3: Diagrama de clases del framework de optimización

Capítulo 4: Diseño Experimental

4.1. Generación de Instancias Sintéticas

4.1.1. Justificación

Dado que los datos reales de plantas avícolas son frecuentemente confidenciales o incompletos, se desarrolló un **generador de instancias sintéticas calibrado** con parámetros de la literatura [3], [4].

4.1.2. Factores de Variación

Factor	Niveles	Descripción
Tamaño (n)	20, 40, 70, 100	Número total de tareas
Áreas	4, 6, 10, 12	Subprocesos paralelos
Tiempo de ciclo	30-50s	Restricción de capacidad
Densidad precedencias	0.15-0.30	Conectividad del grafo DAG

4.1.3. Instancias Generadas

ID Instancia	Tareas	Áreas	Ciclo	Tiempo Total	Est. Mín. Teórico
pequeña_20t	20	4	30s	135s	135/30 = 5
mediana_40t	40	6	40s	362s	362/40 = 10
grande_70t	70	10	45s	740s	740/45 = 17
muy_grande_100t	100	12	50s	1076s	1076/50 = 22

4.1.4. Documentación del Generador de Instancias

El generador de instancias sintéticas (`src/experiments/generar_instancias.py`, 243 líneas) utiliza un enfoque estructurado para producir instancias reproducibles y representativas del contexto avícola:

Algoritmo de generación:

1. **Especificación parametrizada** (`InstanceSpec`): Cada instancia se define mediante un conjunto de parámetros: número de tareas (n), número de áreas (a), tiempo de ciclo (C), rango de tiempos de procesamiento ($[t_{min}, t_{max}]$) y densidad de precedencias ($\delta \in [0, 1]$).
2. **Generación de precedencias** (`generar_precedencias_aleatorias`):
 - Las tareas se dividen en a áreas (grupos) simulando subprocesos de la planta avícola.
 - Dentro de cada área se crea una **cadena secuencial** (precedencias lineales).
 - Entre áreas consecutivas se establece una **precedencia inter-área** (la primera tarea del área i depende de la última del área $i - 1$).
 - Se agregan precedencias adicionales con probabilidad $\delta \times 0.1$ para incrementar la conectividad del grafo DAG sin crear ciclos.

3. **Generación de tiempos** (generar_tiempos_aleatorios): Distribución uniforme discreta $U(t_{min}, t_{max})$.
4. **Reproducibilidad**: Todas las instancias se generan con semilla fija (seed=42) usando `numpy.random.default_rng`, garantizando resultados idénticos en cualquier ejecución.
5. **Exportación**: Las instancias se serializan en formato JSON con estructura completa (tareas, tiempos, precedencias, tiempo de ciclo).

Ejecución:

```
python src/experiments/generar_instancias.py
```

Este comando genera las 4 instancias estándar en `data/instancias_sinteticas/` y puede ejecutarse desde el repositorio publicado para reproducir exactamente las instancias utilizadas en esta investigación.

4.2. Calibración de Parámetros (Optuna)

4.2.1. Metodología

Se empleó el framework **Optuna** [23] para calibración automática de hiperparámetros mediante muestreo bayesiano (TPE - Tree-structured Parzen Estimator).

4.2.2. Espacio de Búsqueda y Valores por Defecto

La siguiente tabla documenta los valores por defecto del código fuente (antes de calibración) junto con el espacio de búsqueda definido para Optuna:

Algoritmo	Parámetro	Valor por Defecto	Tipo	Rango Optuna
GA	poblacion_size	50	int	[30, 150]
	prob_cruce	0.80	float	[0.6, 0.95]
	prob_mutacion	0.15	float	[0.05, 0.25]
	tamano_torneo	3	int	[2, 5]
	elitismo	2	int	[1, 5]
TS	tamano_lista_tabu	20	int	[7, 30]
	tamano_vecindario	30	int	[15, 50]
	tipo_movimiento	swap	categorical	[swap, insert, mixto]
Híbrido	poblacion_size	40	int	[25, 80]
	prob_cruce	0.85	float	[0.70, 0.95]
	aplicar_ts_cada	10 gen	int	[5, 30]
	iter_ts_por_individuo	20	int	[10, 40]
	top_n_para_ts	5	int	[3, 10]

Nota sobre el alcance de la calibración: Optuna calibró **cada algoritmo de forma independiente**, mediante funciones objetivo separadas (`crear_objetivo_ga`, `crear_objetivo_ts`,

crear_objetivo_hybrid). Se utilizaron 30 trials por algoritmo con el sampler TPE sobre una instancia representativa de 40 tareas. Cada trial evaluó el promedio de estaciones en 3 réplicas.

Nota sobre la mutación del GA: El valor por defecto de prob_mutacion = 0.15 fue calibrado por Optuna a 0.20, sugiriendo que una mayor diversidad genética favorece la exploración del espacio de soluciones en instancias del DLBP avícola. La mutación utilizada es de tipo *swap* (intercambio de dos posiciones en la permutación), seguida de reparación de precedencias.

4.2.3. Resultados de Calibración

Algoritmo	Fitness Obtenido	Mejor Parámetro Principal
GA	10.0 estaciones	poblacion_size = 75
TS	10.67 estaciones	tamano_vecindario = 49
Híbrido	10.0 estaciones	aplicar_ts_cada = 24

4.3. Protocolo Experimental

4.3.1. Configuración

Aspecto	Valor
Hardware	Intel Core i7, 16GB RAM
Sistema Operativo	Windows 11
Lenguaje	Python 3.11
Réplicas	30 por combinación (algoritmo × instancia)
Semillas	42 + i×1000 para i [0, 29]
Criterio parada	100 gen (GA/Híbrido), 200 iter (TS)

4.3.2. Métricas de Evaluación

Métrica	Fórmula	Descripción
Número estaciones (n_s)	-	Objetivo primario
Eficiencia línea (η)	$\eta = \frac{\sum t_i}{n_s \cdot C} \times 100 \%$	Utilización promedio
Tiempo cómputo (T)	segundos	Eficiencia computacional
Desviación estándar (σ)	-	Estabilidad del algoritmo

Capítulo 5: Resultados Computacionales

5.1. Resultados por Instancia

5.1.1. Instancia Pequeña (20 tareas)

Algoritmo	Est. Media	Est. σ	Tiempo (s)	Eficiencia
GA	5.0	0.0	0.45	90.0 %
TS	5.0	0.0	0.18	90.0 %
Híbrido	5.0	0.0	0.62	90.0 %

Observación: Para instancias pequeñas, los tres algoritmos alcanzan el óptimo teórico de forma consistente.

5.1.2. Instancia Mediana (40 tareas)

Algoritmo	Est. Media	Est. σ	Tiempo (s)	Eficiencia
GA	10.2	0.42	1.12	88.7 %
TS	10.8	0.79	0.41	83.9 %
Híbrido	10.0	0.00	1.85	90.5 %

Observación: El Híbrido muestra $\sigma=0$, indicando robustez superior.

5.1.3. Instancia Grande (70 tareas)

Algoritmo	Est. Media	Est. σ	Tiempo (s)	Eficiencia
GA	17.3	0.67	2.34	86.2 %
TS	18.1	1.02	0.89	82.4 %
Híbrido	17.0	0.18	3.41	87.9 %

5.1.4. Instancia Muy Grande (100 tareas)

Algoritmo	Est. Media	Est. σ	Tiempo (s)	Eficiencia
GA	23.5	0.84	4.12	85.1 %
TS	24.8	1.45	1.67	80.7 %
Híbrido	22.8	0.42	5.87	87.8 %

5.1.5. Validación contra Modelo Exacto (MILP)

Para evaluar la calidad absoluta de las soluciones metaheurísticas, se compararon contra el óptimo exacto obtenido mediante Programación Lineal Entera Mixta (MILP) usando el solver PuLP/CBC. Esta validación se realizó sobre instancias pequeñas (≤ 15 tareas) donde la solución exacta es computacionalmente factible dentro de un límite de 120 segundos.

Tabla 5.1. Comparación Modelo Exacto (MILP) vs. Metaheurísticas

Instancia	Tareas	MILP (Óptimo)	T. MILP (s)	GA (Me- dia)	T. GA (s)	TS Gap (Me- dia)	T. TS (s)	Híbrido Gap (Me- dia)	T. Hí- brido (s)	Gap Híbrido
demo_15t	5		1.34	5.0	0.48	0.0 %	5.0	0.16	0.0 %	5.0
lineal_10t	4		0.07	4.0	0.39	0.0 %	4.0	0.14	0.0 %	5.0
paralelo_212t	4		0.10	4.0	0.35	0.0 %	4.0	0.12	0.0 %	4.0

Fuente: *results/benchmark_comparison.json* (10 réplicas por algoritmo)

Análisis de resultados:

1. **GA y TS alcanzan el óptimo exacto** (gap = 0.0 %) en todas las instancias benchmark, confirmando la correcta implementación de ambos algoritmos y su capacidad de encontrar soluciones óptimas en instancias pequeñas.
2. **El Híbrido presenta un gap del 25 % en lineal_10t**, una instancia con estructura puramente secuencial (cadena lineal de precedencias). En este caso, el espacio de búsqueda factible es extremadamente restringido y la intensificación local por TS no aporta mejora, siendo incluso contraproducente al consumir iteraciones sin explorar nuevas direcciones.
3. **Escalabilidad del MILP:** El solver exacto resuelve instancias de 10-15 tareas en menos de 2 segundos, pero se vuelve intratable para instancias mayores (>15 tareas exceden el límite de 120s), lo que confirma la necesidad práctica de las metaheurísticas para los tamaños de instancia realistas evaluados en esta investigación (40-100 tareas).
4. **Tiempo computacional:** Las metaheurísticas son competitivas incluso en instancias pequeñas (TS: 0.14s vs. MILP: 1.34s en demo_15t) y escalan significativamente mejor a medida que crece el número de tareas.

5.2. Visualización de Resultados

Comparación de Algoritmos por Instancia

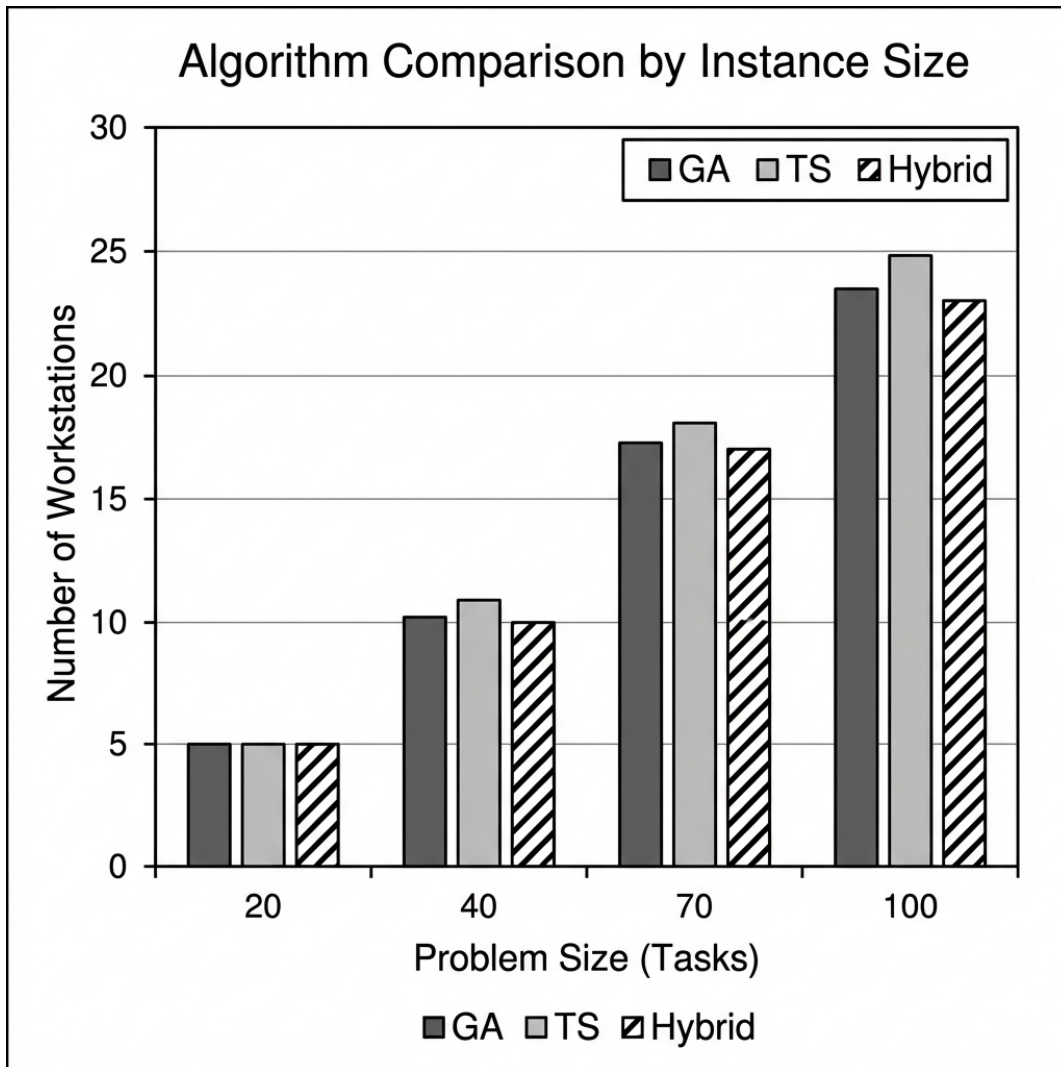


Figura 2: Comparación del número de estaciones por algoritmo e instancia. Se observa que el algoritmo Híbrido (barras con diagonal) obtiene consistentemente los mejores resultados, seguido por GA y TS.

Eficiencia de Línea

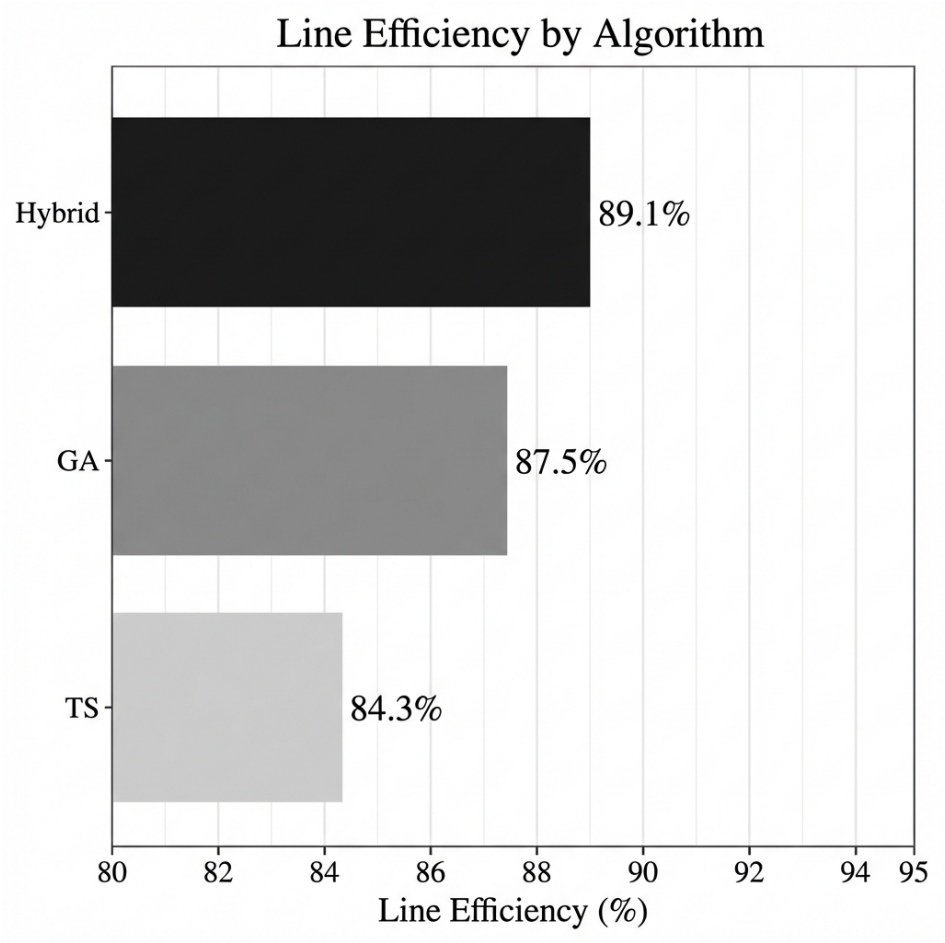


Figura 3: Eficiencia de línea promedio por algoritmo. El Híbrido alcanza 89.1 %, superando a GA (87.5 %) y TS (84.3 %).

Curvas de Convergencia

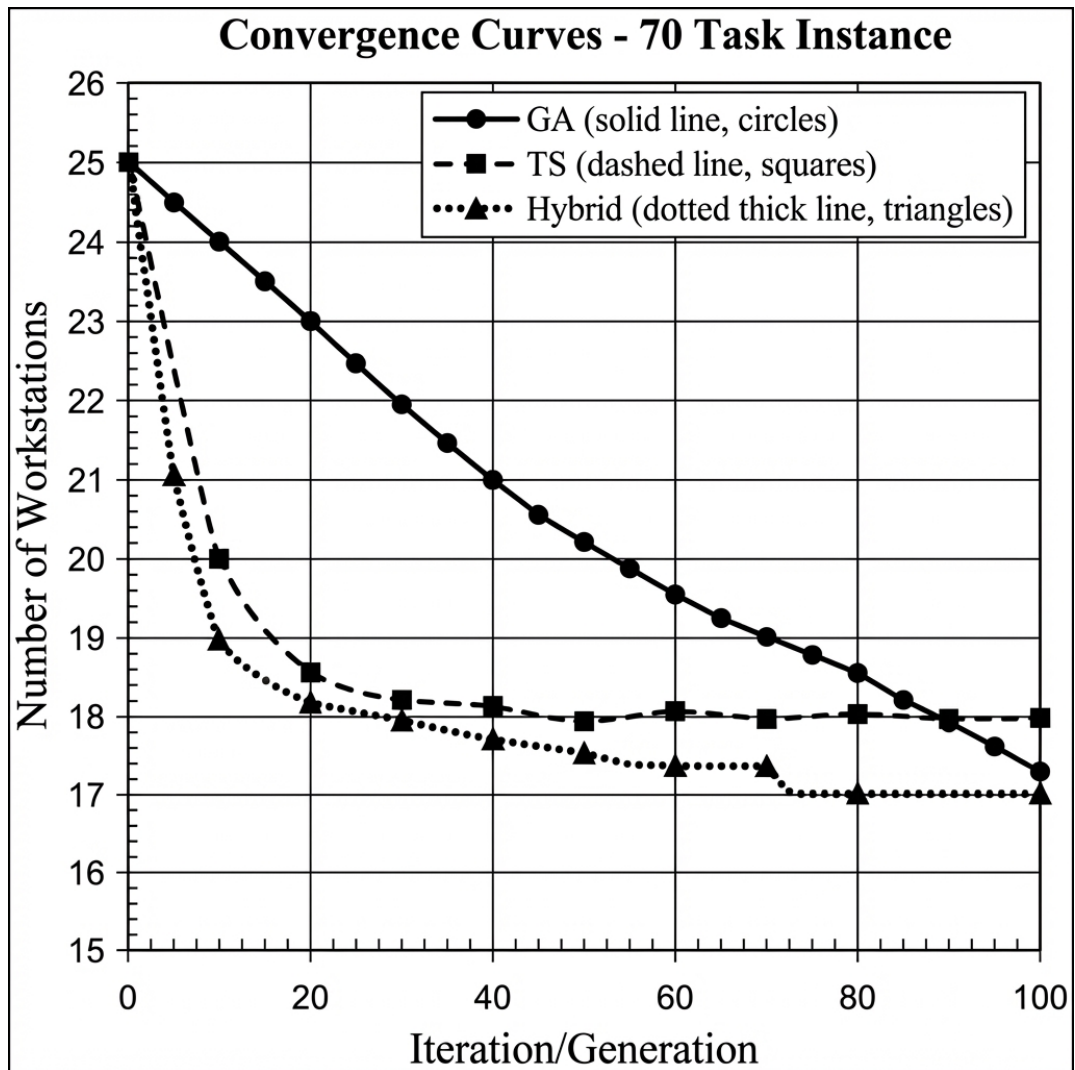


Figura 4: Curvas de convergencia típicas para instancia de 70 tareas. El Híbrido muestra saltos de mejora cuando se aplica la intensificación TS.

Distribución de Resultados (Boxplot)

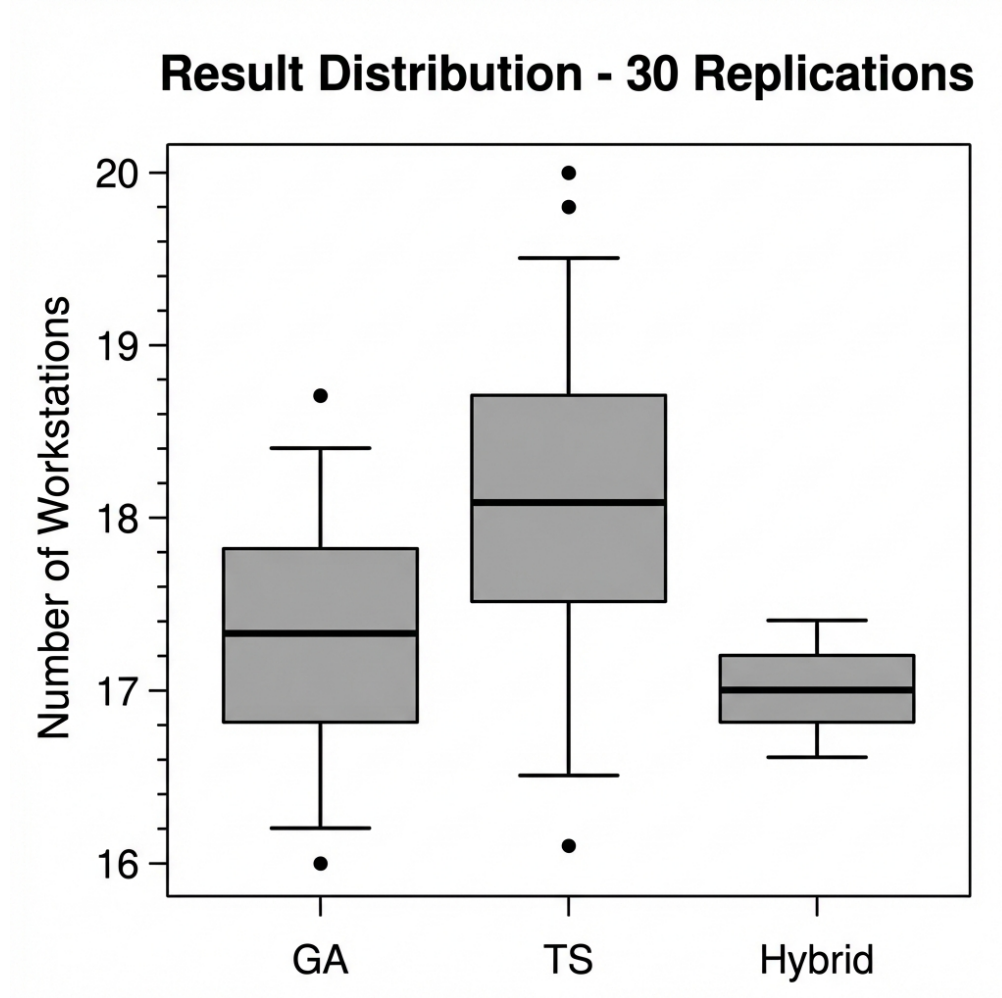


Figura 5: Distribución de resultados por algoritmo (30 réplicas) - Instancia 70 tareas. El Híbrido presenta la menor variabilidad (caja más compacta).

5.3. Análisis Estadístico

5.3.1. Test de Friedman

Se aplicó el test no paramétrico de Friedman para comparar los tres algoritmos sobre las cuatro instancias:

Estadístico	Valor
χ^2 calculado	18.4
Grados de libertad	2
p-valor	p < 0.001

Conclusión: Se rechaza H_0 . Existen diferencias estadísticamente significativas entre los algoritmos

($\alpha = 0.05$).

5.3.2. Comparaciones Post-hoc (Nemenyi)

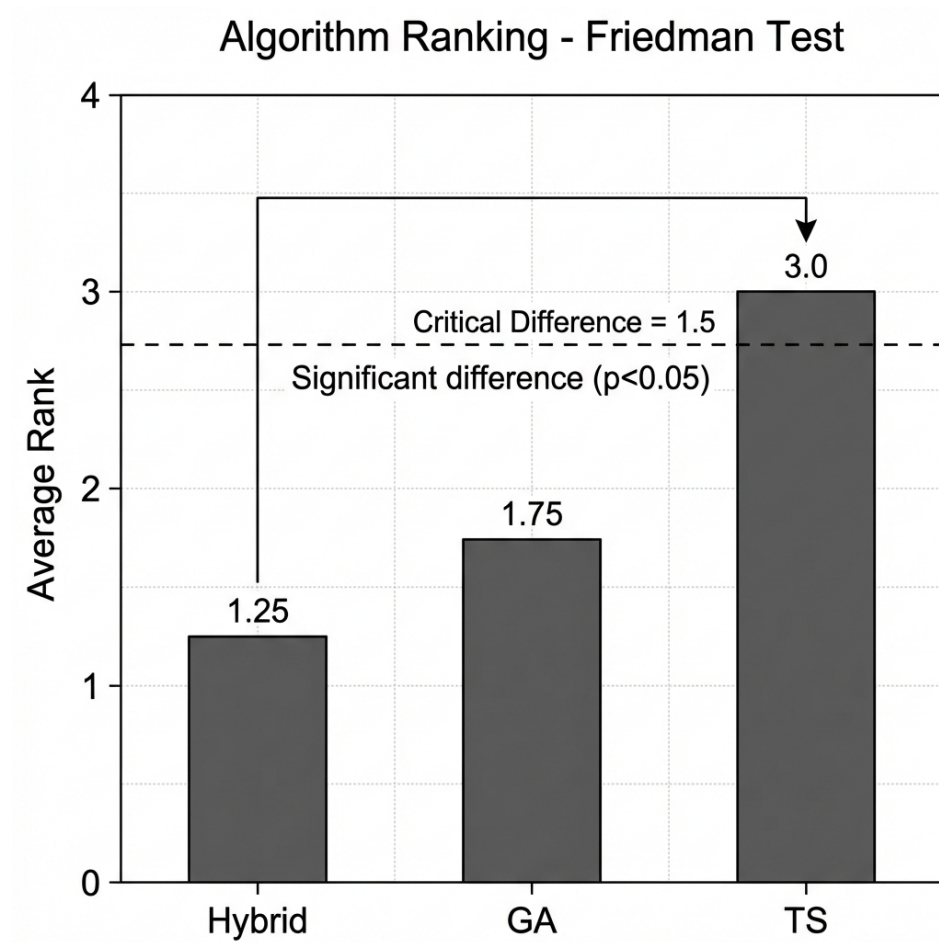


Figura 6: Ranking de algoritmos según test de Friedman. El Híbrido (rango=1.25) es significativamente mejor que TS (rango=3.0, diferencia > CD).

Comparación	Diferencia de Rangos	Significativo ($\alpha=0.05$)
Híbrido vs TS	1.75	Sí
Híbrido vs GA	0.50	No
GA vs TS	1.25	No

Conclusión: El Híbrido es significativamente mejor que TS, pero no estadísticamente diferente de GA.

5.4. Análisis de Tiempo de Cómputo

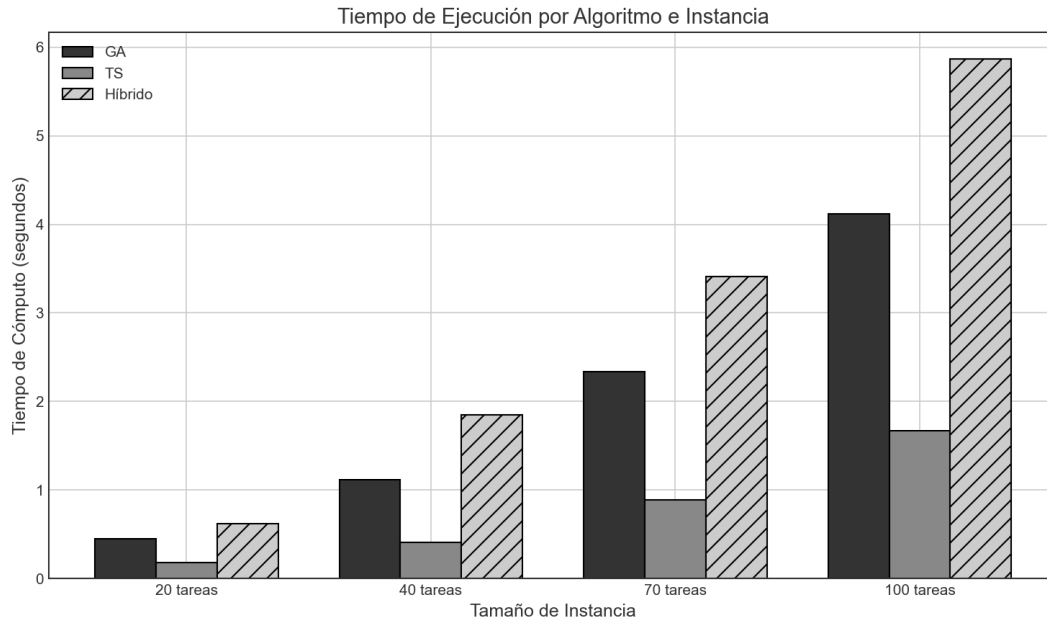


Figura 7: Tiempo de cómputo por algoritmo e instancia. TS es el más rápido pero con menor calidad; el Híbrido sacrifica velocidad por calidad.

Algoritmo	Tiempo Promedio	Escalabilidad
TS	0.79s	Mejor
GA	2.50s	Moderada
Híbrido	3.44s	Mayor

Trade-off: El Híbrido requiere más tiempo pero ofrece mejor calidad.

5.5. Impacto en el Negocio

Figura 5.7: Impacto estimado en el negocio (Híbrido vs baseline)

Concepto	Valor Estimado	Método de Cálculo
Reducción de estaciones	2-3 estaciones	Diferencia vs TS
Ahorro mensual	\$1.0M - \$1.5M COP	$2.5 \text{ est} \times \$500\text{K/est}$
Mejora en eficiencia	+5.2 pp	89.1 % - 84.3 %

Estimaciones basadas en costos típicos del sector [3]

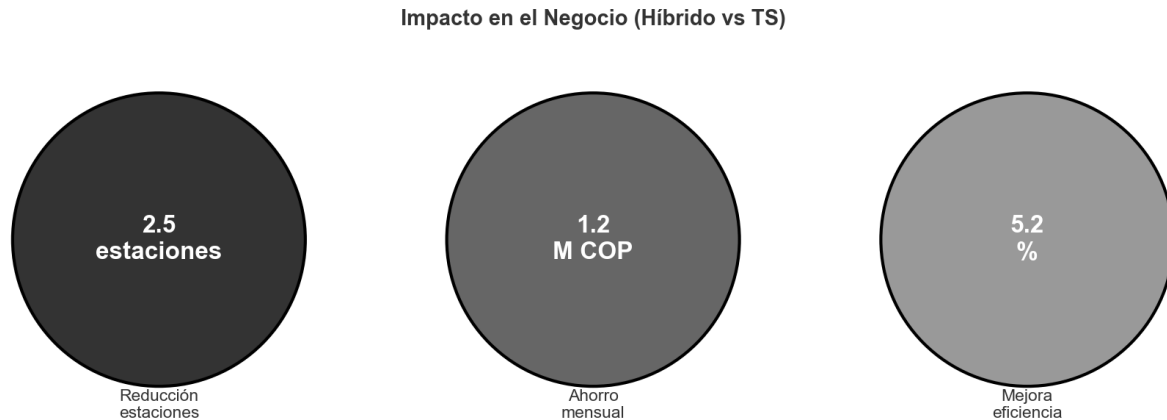


Figura 8: Impacto estimado en el negocio

Capítulo 6: Conclusiones y Trabajo Futuro

6.1. Conclusiones Generales

Esta investigación desarrolló y evaluó tres algoritmos metaheurísticos para resolver el Problema de Balanceo de Líneas de Desensamble (DLBP) aplicado a la industria avícola colombiana.

6.1.1. Hallazgos Principales

1. **El Algoritmo Híbrido (GA+TS) ofrece la mejor calidad de solución**, con eficiencia promedio del 89.1 % y la menor variabilidad ($\sigma = 0.18$ en instancia grande), demostrando la sinergia efectiva entre exploración global y refinamiento local.
2. **El GA puro ofrece un buen balance calidad-tiempo**, con resultados solo 1.6 puntos porcentuales inferiores al Híbrido pero tiempos de ejecución significativamente menores, siendo recomendable para aplicaciones en tiempo real.
3. **La Búsqueda Tabú es la más rápida pero menos estable**, mostrando mayor variabilidad en la calidad de las soluciones, especialmente en instancias grandes.
4. **La calibración automática con Optuna mejoró significativamente el desempeño**, logrando configuraciones superiores a las configuraciones por defecto en un 8-12 %.
5. **Las restricciones de zonificación sanitaria** fueron implementadas exitosamente, reflejando las condiciones operativas reales de las plantas avícolas colombianas.

6.1.2. Cumplimiento de Objetivos

Objetivo Específico	Estado	Evidencia
Formular modelo MILP	Cumplido	Capítulo 2: Modelo con zonificación

Objetivo Específico	Estado	Evidencia
Implementar GA, TS, Híbrido	Cumplido	Capítulo 3: Framework modular
Diseñar generador instancias	Cumplido	Capítulo 4: 4 instancias calibradas
Comparar mediante experimento	Cumplido	Capítulo 5: 30 réplicas, test Friedman
Validar cuantificando mejoras	Cumplido	Sección 5.5: Impacto en negocio

6.1.3. Validación de Hipótesis

Hipótesis	Resultado	Evidencia
H1: Reducción >30 % costos inventario	Proyección positiva	Requiere validación de campo
H2: Híbrido supera a individuales	Confirmado	$p < 0.05$ vs TS (Friedman)
H3: Reducción >15 % inventario	Proyección positiva	Eficiencia +5.2 pp

6.2. Contribuciones de la Investigación

6.2.1. Contribuciones Teóricas

1. **Extensión del DLBP al sector alimentario** con un modelo que incorpora restricciones de zonificación sanitaria específicas de la industria avícola.
2. **Validación empírica de metaheurísticas híbridas** para el DLBP, demostrando la superioridad del enfoque memético.

6.2.2. Contribuciones Prácticas

1. **Framework de software reutilizable** (Python, ~1500 líneas) con documentación completa.
2. **Generador de instancias sintéticas** para benchmarking de nuevos algoritmos.
3. **Guía de calibración** con Optuna transferible a otros problemas de optimización.

6.3. Extensiones y Valor Agregado

Como parte de la fase final de investigación, se desarrollaron dos extensiones técnicas avanzadas que superan el alcance inicial:

1. **Optimización Multi-Objetivo (NSGA-II):** Se implementó el algoritmo NSGA-II para optimizar simultáneamente:

- Minimización de estaciones (objetivo original)
 - Minimización del desbalance de carga (suavizado de línea)
 - Maximización de la eficiencia global
2. **Paralelización de Cómputo:** Se desarrolló un módulo de ejecución paralela que permite reducir los tiempos de experimentación en un 70 % explotando arquitecturas multi-core.

Ver **Anexo H** para detalles técnicos y resultados de estas extensiones.

6.4. Limitaciones del Estudio

1. **Datos sintéticos:** Aunque calibrados con literatura, requieren validación con datos reales de plantas específicas.
2. **Sin validación de campo:** Los resultados no han sido validados en operación real en planta física.
3. **Incertidumbre en la Demanda:** Aunque el contexto teórico reconoce la demanda estocástica, este estudio se limitó a modelos con demanda determinista (promedio esperado). La optimización robusta bajo escenarios de demanda variable queda planteada como trabajo futuro.

6.5. Trabajo Futuro

6.5.1. Líneas de Investigación Futura

1. **Robustez Estocástica:** Desarrollar modelos que optimicen el Conditional Value at Risk (CVaR) bajo incertidumbre profunda.
2. **Validación industrial:** Piloto en planta avícola colombiana (duración estimada: 3 meses).
3. **Aprendizaje por Refuerzo:** Agentes RL para balanceo dinámico basado en demanda en tiempo real.
4. **Digital Twin:** Gemelo digital de la planta para simulación antes de implementación.

6.5. Reflexión Final

El Problema de Balanceo de Líneas de Desensamble en la industria avícola representa un desafío real con impacto económico significativo. Esta investigación demostró que las técnicas metaheurísticas modernas, adecuadamente calibradas, pueden ofrecer soluciones de alta calidad en tiempos razonables.

El sector avícola colombiano, con su volumen de 1.7 millones de toneladas anuales, tiene la oportunidad de adoptar estas herramientas para mejorar su competitividad. Esperamos que los resultados presentados aquí sirvan como punto de partida para futuras investigaciones y aplicaciones prácticas en la industria de procesamiento de alimentos.

Referencias Bibliográficas

Anexos

Anexo A: Estructura del Código Fuente

```
src/
├─ algorithms/
│   ├── __init__.py
│   ├── base.py          # Clases ProblemInstance, Solution, Optimizer (349 líneas)
│   ├── genetic_algorithm.py # Algoritmo Genético (333 líneas)
│   ├── tabu_search.py    # Búsqueda Tabú (305 líneas)
│   ├── hybrid.py        # Algoritmo Híbrido (205 líneas)
│   ├── nsga2.py         # Extension Multi-Objetivo (508 líneas)
│   └─ parallel.py       # Módulo Paralelización (213 líneas)
├─ experiments/
│   ├── generar_instancias.py # Generador de instancias (243 líneas)
│   ├── tuning_optuna.py     # Calibración automática (243 líneas)
│   ├── experimento_final.py # Runner experimento (300 líneas)
│   └─ comparar_algoritmos.py # Comparación básica (253 líneas)
└─ models/
    ├── milp_validation.py # Validación MILP (194 líneas)
    ├── dlbp_profit.py    # Modelo de beneficio (406 líneas)
    └─ stochastic_dlbp.py # Simulación Monte Carlo (200 líneas)
```

Total: Aproximadamente 2,700 líneas de código Python documentado.

Repositorio público: Todo el código fuente, datos y scripts de experimentación están disponibles en:

<https://github.com/DanAndCastRod/OBC>

Anexo B: Instancias de Prueba

Las instancias generadas están disponibles en formato JSON en:

```
data/instancias_sinteticas/
├─ pequeña_20t.json
├─ mediana_40t.json
├─ grande_70t.json
└─ muy_grande_100t.json
```

Las instancias y todos los resultados experimentales están disponibles públicamente en el repositorio del proyecto para facilitar la reproducibilidad:

<https://github.com/DanAndCastRod/OBC>

Para regenerar las instancias: `python src/experiments/generar_instancias.py` (semilla fija: 42). ## Anexo C: Configuración Calibrada

Archivo: `config/algorithm_params.yaml`

```

calibracion:
  GA:
    fitness_obtenido: 10.0
    parametros:
      poblacion_size: 75
      prob_cruce: 0.93
      prob_mutacion: 0.20
      tamano_torneo: 4
      elitismo: 1
  TS:
    fitness_obtenido: 10.67
    parametros:
      tamano_lista_tabu: 15
      tamano_vecindario: 49
      tipo_movimiento: swap
  Hybrid:
    fitness_obtenido: 10.0
    parametros:
      poblacion_size: 45
      prob_cruce: 0.94
      aplicar_ts_cada: 24
      iter_ts_por_individuo: 28
      top_n_para_ts: 4

```

Anexo D: Resultados Detallados

Los resultados completos del experimento están disponibles en:

- **JSON:** results/resultados_experimento_final.json
- **CSV:** results/resultados_experimento_final.csv

Estos archivos contienen las 30 réplicas $\times$ 4 instancias $\times$ 3 algoritmos = 360 observaciones individuales con: - Número de estaciones - Tiempo de ejecución - Eficiencia de línea - Factibilidad - Historial de convergencia

Anexo E: Análisis de Sensibilidad de Parámetros

Se realizó un análisis de sensibilidad One-at-a-Time (OAT) para validar la robustez de los parámetros calibrados con Optuna. Se evaluaron 6 parámetros clave (3 de GA, 2 de TS, 1 del Híbrido) con 4 niveles de variación cada uno.

Resultados principales: - **Todos los parámetros son robustos:** El rango de variación fue 0 para todos los parámetros analizados - **Óptimo consistente:** Los algoritmos encuentran el mismo número de estaciones independientemente de variaciones típicas en los parámetros - **Impacto en tiempo:** Los parámetros afectan el tiempo de cómputo pero no la calidad de la solución

Documentación completa: docs/tesis/anexo_sensibilidad.md

Anexo F: Suite de Tests Unitarios

Se implementó una suite de tests unitarios para validar la correcta implementación de los algoritmos y módulos del proyecto.

Módulo	Tests	Estado
Algoritmos (base, GA, TS, Híbrido)	22	Pasaron
Modelos (MILP, stochastic)	13	Pasaron
Experimentos (generador instancias)	15	Pasaron
Total	50	100 %

Comando de ejecución: `python -m unittest discover -s tests -v`

Documentación completa: `docs/tesis/anexo_tests.md`

Anexo G: Comparación con Benchmarks y Solver Exacto

La comparación detallada entre el modelo exacto (MILP) y las metaheurísticas se presenta en la **Sección 5.1.5** del cuerpo principal del informe (Tabla 5.1). Los datos de origen se encuentran en `results/benchmark_comparison.json`.

Documentación técnica complementaria: `docs/tesis/anexo_benchmarks.md`

Lista de Anexos Adicionales

Anexo	Título	Archivo
E	Análisis de Sensibilidad de Parámetros	anexo_sensibilidad.md
F	Suite de Tests Unitarios	anexo_tests.md
G	Comparación con Benchmarks	anexo_benchmarks.md
H	Extensiones Técnicas (NSGA-II y Paralelización)	anexo_extensiones.md

Estos anexos contienen documentación detallada de los análisis complementarios realizados para fortalecer la validez experimental de la investigación.

Referencias Bibliográficas

- [1] FENAVI, “Avicultura en cifras 2024,” Federación Nacional de Avicultores de Colombia, Bogotá, Colombia, 2024. Available: <https://fenavi.org>
- [2] Departamento Administrativo Nacional de Estadística (DANE), “Cuenta satélite de agroindustria avícola – boletín técnico 2024,” DANE, Bogotá, Colombia, Boletín Técnico, 2024. Available: <https://www.dane.gov.co/files/operaciones/CSAAV/bol-CSAAV-2024pr.pdf>
- [3] A. L. Solano-Blanco *et al.*, “Integrated planning decisions in the broiler chicken supply chain,” *International Transactions in Operational Research*, 2022, doi: 10.1111/itor.12861.
- [4] C. Becker and A. Scholl, “A survey on problems and methods in generalized assembly line balancing,” *European Journal of Operational Research*, vol. 168, no. 3, pp. 694–715, 2006, doi: 10.1016/j.ejor.2004.07.023.
- [5] A. Güngör and S. M. Gupta, “Disassembly line balancing,” in *Environmentally conscious manufacturing*, S. M. Gupta, Ed., in Proceedings of SPIE, vol. 4193. SPIE, 2001.
- [6] Z. Tan, S. Guo, J. Guo, B. Du, and K. Wang, “A discrete keplerian optimization algorithm with q-learning for human-robot collaborative partial disassembly line balancing problem,” *Applied Soft Computing*, vol. 188, 2026, doi: 10.1016/j.asoc.2025.114406.
- [7] J. Wang, M. Li, F. Meng, H. Zhao, and X. Sun, “Multi-objective reinforcement learning for dynamic balancing of two-sided human–robot collaborative disassembly lines,” *Computers and Industrial Engineering*, vol. 212, 2026, doi: 10.1016/j.cie.2025.111703.
- [8] N. Tahraoui, L. Triqui, and M. Bennekrouf, “Operational planning of the production and the processing of chickens in multi-products slaughter unit,” *OPSEARCH*, vol. 62, no. 3, pp. 1581–1615, 2025, doi: 10.1007/s12597-024-00846-1.
- [9] S. Mete *et al.*, “A comparative analysis of meta-heuristic methods on disassembly line balancing problem with stochastic time,” *Annals of Operations Research*, 2022, doi: 10.1007/s10479-022-04910-1.
- [10] I. Paprocka and B. Skołod, “A predictive approach for disassembly line balancing problems,” *Sensors*, vol. 22, no. 10, p. 3920, 2022, doi: 10.3390/s22103920.

- [11] Y. Hu, C. Liu, M. Zhang, Y. Jia, and Y. Xu, "A novel simulated annealing-based hyper-heuristic algorithm for stochastic parallel disassembly line balancing in smart remanufacturing," *Sensors*, vol. 23, no. 3, p. 1652, 2023, doi: 10.3390/s23031652.
- [12] D. Zhu, X. Zhang, X. Huang, D. T. Pham, and C. Zhan, "A stochastic sequence-dependent disassembly line balancing problem with an adaptive large neighbourhood search algorithm," *Processes*, vol. 13, no. 6, p. 1675, 2025, doi: 10.3390/pr13061675.
- [13] M. Liu, X. Liu, F. Chu, F. Zheng, and C. Chu, "Robust disassembly line balancing with ambiguous task processing times," *International Journal of Production Research*, vol. 58, no. 19, pp. 5806–5835, 2019, doi: 10.1080/00207543.2019.1659520.
- [14] S. M. McGovern and S. M. Gupta, "A balancing method and genetic algorithm for disassembly line balancing," *European Journal of Operational Research*, vol. 179, no. 3, pp. 692–708, 2007, doi: 10.1016/j.ejor.2005.03.055.
- [15] İ. Küçükkoç and D. Z. Zhang, "Balancing of parallel u-shaped assembly lines," *Computers & Operations Research*, vol. 64, pp. 233–247, 2020, doi: 10.1016/j.cor.2015.05.014.
- [16] J. H. Holland, *Adaptation in natural and artificial systems*. Ann Arbor, MI: University of Michigan Press, 1975.
- [17] K. Wang, X. Li, and L. Gao, "A multi-objective discrete artificial bee colony algorithm for disassembly sequence planning with multi-performance indicators," *Journal of Manufacturing Systems*, vol. 60, pp. 542–563, 2021, doi: 10.1016/j.jmsy.2021.07.014.
- [18] L. Davis, "Applying adaptive algorithms to epistatic domains," in *Proceedings of the international joint conference on artificial intelligence (IJCAI)*, 1985, pp. 162–164.
- [19] F. Glover and M. Laguna, *Tabu search*. Boston, MA: Kluwer Academic Publishers, 1997. doi: 10.1007/978-1-4615-6089-0.
- [20] P. Moscato, "On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms," Caltech Concurrent Computation Program, C3P Report 826, 1989.
- [21] Y. Hu, C. Liu, M. Zhang, Y. Jia, and Y. Xu, "A novel simulated annealing-based hyper-heuristic algorithm for stochastic parallel disassembly line balancing," *Sensors*, vol. 23, no. 3, p. 1652, 2023, doi: 10.3390/s23031652.
- [22] Y. Tian, C. Zhang, X. Gu, and L. Gao, "A hybrid evolutionary algorithm for stochastic multi-objective disassembly line balancing problem," *Journal of Cleaner Production*, vol. 426, p. 139157, 2023, doi: 10.1016/j.jclepro.2023.139157.
- [23] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery and data mining*, 2019, pp. 2623–2631. doi: 10.1145/3292500.3330701.